

WEEK_6

Catalogs and Orders Microservice

Todo

- Users Microservice 기능 추가
- Catalogs Microservice 프로젝트 생성
- Orders Microservice 프로젝트 생성

Users Microservice - 사용자 조회 (1)

Features

- ☒ 신규 회원 등록
- ☐ 회원 로그인
- ☐ 상세 정보 확인
- ☐ 회원 정보 수정/삭제
- ☐ 상품 주문
- ☐ 주문 내역 확인

APIs

기능	URI (API Gateway 사용 시)	URI (API Gateway 미사용 시)	HTTP Method
사용자 정보 등록	/user-service/users	/users	POST
전체 사용자 조회	/user-service/users	/users	GET
사용자 정보, 주문 내역 조회	/user-service/users/{user_id}	/users/{user_id}	GET
작동 상태 확인	/user-service/users/health-check	/health-check	GET
환영 메시지	/user-service/users/welcome	/welcome	GET

API Gateway Service 변경

- User Service route 추가 (포트번호 대신 서비스 이름, user-service으로 쉽게 접근하기 위함)

```
spring:
  application:
    name: apigateway-service
  cloud:
    gateway:
      server:
        webflux:
          default-filters:
            - name: GlobalFilter
              args:
                baseMessage: Spring Cloud Gateway WebFlux Global Filter
                preLogger: true
                postLogger: true
          routes:
            - id: user-service
              uri: lb://USER-SERVICE
              predicates:
                - Path=/user-service/**
            - id: first-service
              uri: lb://MY-FIRST-SERVICE
              predicates:
                - Path=/first-service/**
              filters:
                - AddRequestHeader=f-request, 1st-request-header-by-yaml
                - AddResponseHeader=first-response, 1st-response-header-from-yaml
                - CustomFilter
            - id: second-service
```

서버 실행

- Eureka Server 실행 → API Gateway Service, User Service 실행 → localhost:8761 접속

Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
APIGATEWAY-SERVICE	n/a (1)	(1)	UP (1) - 192.168.25.20:apigateway-service:8000
USER-SERVICE	n/a (1)	(1)	UP (1) - user-service:9ecaec22c850e6254e68802e48a9b59e

Users Microservice와 Spring Cloud Gateway 연동

Whitelabel Error Page

This application has no configured error view, so you are seeing this as a fallback.

Tue Oct 14 23:43:54 KST 2025
[7acddb5c-5] There was an unexpected error (type=Not Found, status=404).
No static resource user-service/welcome.
org.springframework.web.reactive.resource.NoResourceFoundException: 404 NOT_FOUND "No static resource user-service/welcome."
at org.springframework.web.reactive.resource.ResourceWebHandler.lambda\$handle\$1(ResourceWebHandler.java:432)
Suppressed: The stacktrace has been enhanced by Reactor, refer to additional information below:
Error has been observed at the following site(s):
* __checkpoint → org.springframework.cloud.gateway.filter.WeightCalculatorWebFilter [DefaultWebFilterChain]
* __checkpoint → HTTP GET "/user-service/welcome" [ExceptionHandlerWebHandler]

<http://localhost:8000/user-service/welcome> 호출

현재 구조에서 API Gateway를 통해 User Service를 호출하면 문제가 발생함. 원인은 User Service의 URI와 API Gateway의 URI가 서로 다르기 때문.

```
routes:
  - id: user-service
    uri: lb://USER-SERVICE
    predicates:
      - Path=/user-service/**
```

API Gateway 설정(application.yml)을 확인해보면, /user-service/**로 들어오는 모든 요청은 유레카에 등록된 USER-SERVICE로 포워딩되도록 되어 있음. (현재 포워딩까지는 수행 되는 중)

```
bum0w0
@GetMapping("/welcome")
public String welcome(HttpServletRequest request) {
    log.info("users.welcome ip: {}, {}, {}, {}", request.getRemoteAddr()
        , request.getRemoteHost(), request.getRequestURI(), request.getRequestURL());

    // return env.getProperty("greeting.message");
    return greeting.getMessage();
}
```

그런데 User Service의 컨트롤러에서는 API Gateway가 전달해주는 user-service/welcome 요청을 /welcome으로만 처리하고 있어서, URI가 일치하지 않아 문제가 발생하는 것

Root Request Mapping 정보 변경 (User Service)

- UserController.java 수정

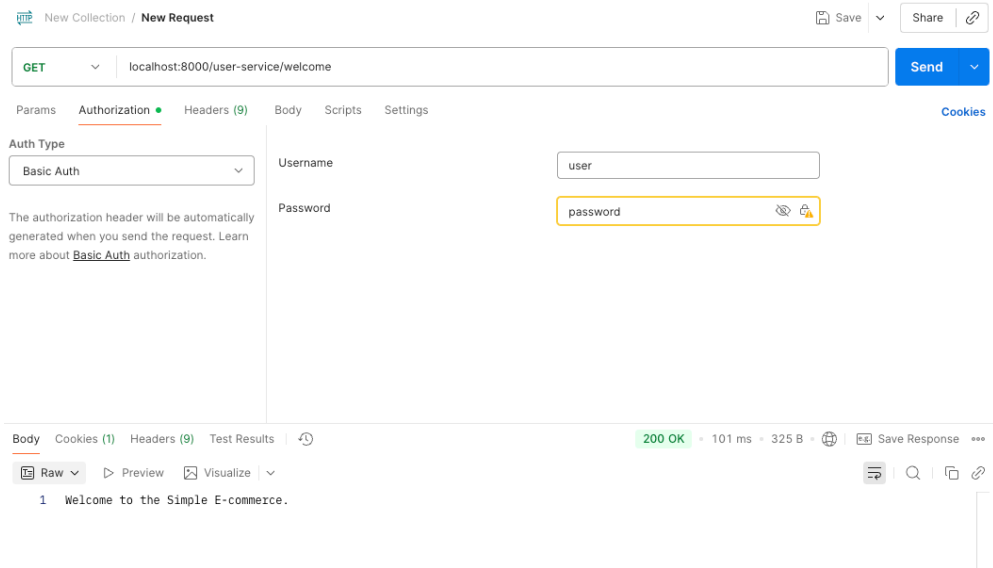
```
// Gateway에서 들어오는 /user-service/** 요청을 처리
@RequestMapping("/user-service")

@GetMapping("/health-check")
public String status() {
    return String.format("It's Working in User Service"
        + ", port(local.server.port)=" + env.getProperty("local.server.port")
        + ", port(server.port)=" + env.getProperty("server.port"));
}

@GetMapping("/welcome")
public String welcome(HttpServletRequest request) {
    log.info("users.welcome ip: {}, {}, {}, {}", request.getRemoteAddr()
        , request.getRemoteHost(), request.getRequestURI(), request.getRequestURL());

    // return env.getProperty("greeting.message");
    return greeting.getMessage();
}
```

API Gateway를 통해 다시 호출 : localhost:8000/user-service/welcome



Gateway 연동 성공

Users Microservice - 사용자 조회 (2)

Response Data 추가

- ResponseUser.java

```
@Data
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ResponseUser {
    private String email;

    private String name;

    private String userId;

    private List<ResponseOrder> orders;
}
```

- ResponseOrder.java

```
@Data
public class ResponseOrder {
    private String productId;
    private Integer qty;
    private Integer unitPrice;
    private Integer totalPrice;
    private Date createdAt;

    private String orderId;
}
```

UserService Method 추가

- UserService.java

```
public interface UserService {  
    UserDto createUser(UserDto userDto);  
    UserDto getUserById(String userId);  
    public Iterable<UserEntity> getUserByAll();  
}
```

- UserServiceImpl.java

```
@Override  
public UserDto getUserById(String userId) {  
    UserEntity userEntity = userRepository.findById(userId);  
  
    if (userEntity == null) {  
        throw new RuntimeException("User not found");  
    }  
  
    UserDto userDto = new ModelMapper().map(userEntity, UserDto.class);  
  
    List<ResponseOrder> orderList = new ArrayList<>();  
    userDto.setOrders(orderList);  
  
    return userDto;  
}  
  
@Override  
public Iterable<UserEntity> getUserByAll() {  
    return userRepository.findAll();  
}
```

UserController 엔드포인트 추가

```
@GetMapping("/users")  
public ResponseEntity getUsers() {  
    Iterable<UserEntity> userList = userService.getUserByAll();  
  
    return ResponseEntity.status(HttpStatus.OK).body(userList);  
}  
  
@GetMapping("/users/{userId}")  
public ResponseEntity getUser(@PathVariable("userId") String userId) {  
    UserDto userDto = userService.getUserById(userId);  
    ResponseUser returnValue = new ModelMapper().map(userDto, ResponseUser.class);  
  
    return ResponseEntity.status(HttpStatus.OK).body(returnValue);  
}
```

Users Microservice - 사용자 조회 (3)

회원 전체 조회

New Collection / New Request Copy 2

GET localhost:8000/user-service/users

Send

Params Authorization Headers (11) Body Scripts Settings

Headers 10 hidden

Key	Value	Description
Authorization	Bearer eyJhbGciOiJIUzI1NiJ9.eyJlbWVpbC6InVzZXI2...	
Key	Value	Description

Body Cookies (1) Headers (9) Test Results

200 OK 110 ms 645 B

JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "email": "kgy5988@naver.com",
5     "name": "KIM",
6     "userId": "8c24d18f-4818-493c-8e1b-df1bdf33f40a",
7     "encryptedPwd": "$2a$10$z9Suy0w4pvh29UFndt0TB.PNjxKVJuk9wrICuSqWHG1uE39gI1qW5"
8   },
9   {
10    "id": 5,
11    "email": "kgy5988@gmail.com",
12    "name": "KIM",
13    "userId": "64be0af9-63e6-4af9-829c-d826c1d31593",
14    "encryptedPwd": "$2a$10$VLPV4jCshVsqLQeAvWyQ.6gc1/EmaIn41K7NcXCUGpWEFVvyX6fG"
15  }
16 ]
```

회원 상세 조회

New Collection / New Request Copy 3

GET localhost:8000/user-service/users/df559268-d85f-4042-9767-3ce35e7096fc

Send

Params Authorization Headers (11) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies (1) Headers (9) Test Results

200 OK 189 ms 394 B

JSON Preview Visualize

```
1 {
2   "email": "kgy5988@gmail.com",
3   "name": "KIM",
4   "userId": "df559268-d85f-4042-9767-3ce35e7096fc",
5   "orders": []
6 }
```

Catalogs Microservice - 개요

Features

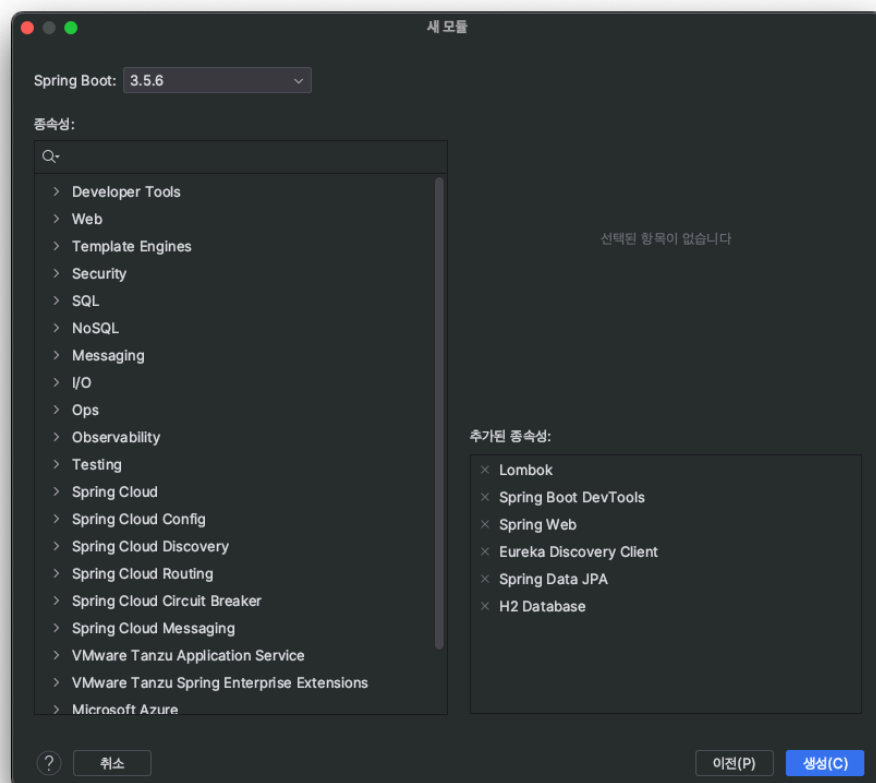
- ☒ 신규 회원 등록
- ☐ 회원 로그인
- ☒ 상세 정보 확인
- ☐ 회원 정보 수정/삭제
- ☐ 상품 주문
- ☒ 주문 내역 확인

APIs

기능	마이크로서비스	URI (API Gateway 사용 시)	HTTP Method
상품 목록 조회	Catalogs Microservice	/catalog-service/catalogs	GET
사용자 별 상품 주문	Orders Microservice	/order-service/{user_id}/orders	POST
사용자 별 주문 내역 조회	Orders Microservice	/order-service/{user_id}/orders	GET

프로젝트 생성 (catalog-service)

- dependencies



- application.yml 파일

```

1  server:
2    port: 0
3
4  spring:
5    application:
6      name: catalog-service
7    h2:
8      console:
9        enabled: true
10       settings:
11         web-allow-others: true
12       path: /h2-console
13    datasource:
14      driver-class-name: org.h2.Driver
15      url: jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;
16      username: sa
17      password:
18    jpa:
19      hibernate:
20        ddl-auto: create-drop
21      show-sql: true
22      generate-ddl: true
23      database: h2
24      defer-datasource-initialization: true
25
26  eureka:
27    instance:
28      instance-id: ${spring.application.name}:${spring.application.instance_id:${random.value}}
29    client:
30      service-url:
31        defaultZone: http://127.0.0.1:8761/eureka
32      fetch-registry: true
33      register-with-eureka: true
34
35  logging:
36    level:
37      org.springframework.security: DEBUG

```

Catalogs Microservice - 기능 구현 (1)

- data sql 추가

```

insert into catalog(product_id, product_name, stock, unit_price) values('CATALOG-001', 'Berlin', 100, 150
0);
insert into catalog(product_id, product_name, stock, unit_price) values('CATALOG-002', 'Tokyo', 110, 100
0);
insert into catalog(product_id, product_name, stock, unit_price) values('CATALOG-003', 'Stockholm', 120,
2000);

```

- Entity, Repository 추가

```

package com.example.catalogservice.jpa;

import jakarta.persistence.*;
import lombok.Data;
import org.hibernate.annotations.ColumnDefault;

import java.util.Date;

@Data

```



```

@Entity
@Table(name = "catalog")
public class CatalogEntity { // implements Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120, unique = true)
    private String productId;
    @Column(nullable = false)
    private String productName;
    @Column(nullable = false)
    private Integer stock;
    @Column(nullable = false)
    private Integer unitPrice;

    @Column(nullable = false, updatable = false, insertable = false)
    @ColumnDefault(value = "CURRENT_TIMESTAMP")
    private Date createdAt;
}

```

implements Serializable → 객체를 파일, 네트워크 전송, 캐시 등에 저장하거나 주고받을 수 있도록 "직렬화 가능한 객체"로 표시하는 것

```

public interface CatalogRepository extends CrudRepository<CatalogEntity, Long> {
    CatalogEntity findByProductId(String productId);
}

```

• DTO, VO 추가

```

@Data
public class CatalogDto { // implements Serializable {
    private String productId;
    private Integer qty;
    private Integer unitPrice;
    private Integer totalPrice;

    private String orderId;
    private String userId;
}

```

```

@Data
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ResponseCatalog {
    private String productId;
    private String productName;
    private Integer unitPrice;
    private Integer stock;
    private Date createdAt;
}

```

- **Service 추가**

```
public interface CatalogService {  
    Iterable<CatalogEntity> getAllCatalogs();  
}
```

```
@Data  
@Slf4j  
@Service  
public class CatalogServiceImpl implements CatalogService {  
    CatalogRepository catalogRepository;  
  
    Environment env;  
  
    @Autowired  
    public CatalogServiceImpl(CatalogRepository catalogRepository, Environment env) {  
        this.catalogRepository = catalogRepository;  
        this.env = env;  
    }  
  
    @Override  
    public Iterable<CatalogEntity> getAllCatalogs() {  
        return catalogRepository.findAll();  
    }  
}
```

- **Controller 추가**

```
@RestController  
@RequestMapping("/catalog-service")  
public class CatalogController {  
    Environment env;  
    CatalogService catalogService;  
  
    @Autowired  
    public CatalogController(Environment env, CatalogService catalogService) {  
        this.env = env;  
        this.catalogService = catalogService;  
    }  
  
    @GetMapping("/health-check")  
    public String status() {  
        return String.format("It's Working in Catalog Service on LOCAL PORT %s (SERVER PORT %s)",  
            env.getProperty("local.server.port"),  
            env.getProperty("server.port"));  
    }  
  
    @GetMapping("/catalogs")  
    public ResponseEntity<List<ResponseCatalog>> getCatalogs() {  
        Iterable<CatalogEntity> catalogList = catalogService.getAllCatalogs();  
  
        List<ResponseCatalog> result = new ArrayList<>();  
        catalogList.forEach(v → {
```

```

        result.add(new ModelMapper().map(v, ResponseCatalog.class));
    });

    return ResponseEntity.status(HttpStatus.OK).body(result);
}
}

```

Catalogs Microservice - 기능 구현 (2)

- API Gateway Service에 Catalog Service 등록

```

spring:
  application:
    name: apigateway-service
  cloud:
    gateway:
      server:
        webflux:
          default-filters:
            - name: GlobalFilter
              args:
                baseMessage: Spring Cloud Gateway WebFlux Global Filter
                preLogger: true
                postLogger: true
          routes:
            - id: user-service
              uri: lb://USER-SERVICE
              predicates:
                - Path=/user-service/**
            - id: catalog-service
              uri: lb://CATALOG-SERVICE
              predicates:
                - Path=/catalog-service/**
            - id: first-service
              uri: lb://MY-FIRST-SERVICE
              predicates:
                - Path=/first-service/**
              filters:
                - AddRequestHeader=f-request, 1st-request-header-by-yaml
                - AddResponseHeader=first-response, 1st-response-header-from-yaml
                - CustomFilter
            - id: second-service
              uri: lb://MY-SECOND-SERVICE
              predicates:
                - Path=/second-service/**
              filters:
                - AddRequestHeader=s-request, 2nd-request-header-by-yaml
                - AddResponseHeader=s-response, 2nd-response-header-from-yaml

```

- 서버 실행 및 API 테스트

GET http://localhost:8000/catalog-service/catalogs Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

Body Cookies Headers (3) Test Results 200 OK 833 ms 491 B Save Response

{ } JSON Preview Visualize

```

1  [
2    {
3      "productId": "CATALOG-001",
4      "productName": "Berlin",
5      "unitPrice": 1500,
6      "stock": 100,
7      "createdAt": "2025-10-15T15:16:02.569+00:00"
8    },
9    {
10     "productId": "CATALOG-002",
11     "productName": "Tokyo",
12     "unitPrice": 1000,
13     "stock": 110,
14     "createdAt": "2025-10-15T15:16:02.570+00:00"
15   },
16   {
17     "productId": "CATALOG-003",
18     "productName": "Stockholm",
19     "unitPrice": 2000,
20     "stock": 120,
21     "createdAt": "2025-10-15T15:16:02.570+00:00"
22   }
23 ]

```

Orders Microservice - 개요

Features

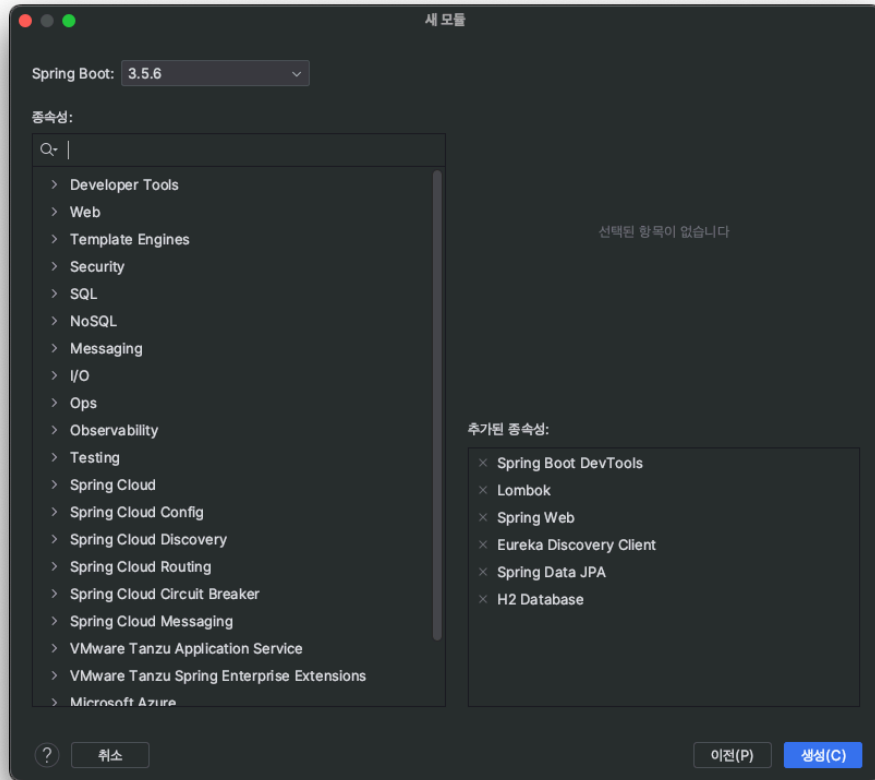
- ☒ 상품 목록 조회
- ☐ 사용자 별 상품 주문
- ☐ 사용자 별 주문 내역 조회

APIs

기능	마이크로서비스	URI (API Gateway 사용 시)	HTTP Method
상품 목록 조회	Catalogs Microservice	/catalog-service/catalogs	GET
사용자 별 상품 주문	Orders Microservice	/order-service/{user_id}/orders	POST
사용자 별 주문 내역 조회	Orders Microservice	/order-service/{user_id}/orders	GET

프로젝트 생성 (order-service)

- dependencies



- application.yml 파일

```

1  server:
2    port: 0
3
4  spring:
5    application:
6      name: order-service
7    h2:
8      console:
9        enabled: true
10       settings:
11         web-allow-others: true
12       path: /h2-console
13    datasource:
14      driver-class-name: org.h2.Driver
15      url: jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;
16      username: sa
17      password:
18    jpa:
19      hibernate:
20        ddl-auto: update
21
22  eureka:
23    instance:
24      instance-id: ${spring.application.name}:${spring.application.instance_id:${random.value}}
25    client:
26      service-url:
27        defaultZone: http://127.0.0.1:8761/eureka
28      fetch-registry: true
29      register-with-eureka: true
30
31  logging:
32    level:
33      org.springframework.security: DEBUG

```

Orders Microservice - 기능 구현 (1)

- Entity, Repository 추가

```

@Data
@Entity
@Table(name="orders")
public class OrderEntity implements Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 120)
    private String productId;
    @Column(nullable = false)
    private Integer qty;
    @Column(nullable = false)
    private Integer unitPrice;
    @Column(nullable = false)
    private Integer totalPrice;

    @Column(nullable = false)
    private String userId;
    @Column(nullable = false, unique = true)
    private String orderId;

```

```

@Column(nullable = false, updatable = false, insertable = false)
@ColumnDefault(value = "CURRENT_TIMESTAMP")
private Date createdAt;
}

```

```

public interface OrderRepository extends CrudRepository<OrderEntity, Long> {
    OrderEntity findById(String orderId);
    Iterable<OrderEntity> findByUserId(String userId);
}

```

- **DTO, VO 추가**

```

@Data
public class OrderDto implements Serializable {
    private String productId;
    private Integer qty;
    private Integer unitPrice;
    private Integer totalPrice;

    private String orderId;
    private String userId;
}

```

```

@Data
public class RequestOrder {
    private String productId;
    private Integer qty;
    private Integer unitPrice;
}

```

```

@Data
@JsonInclude(JsonInclude.Include.NON_NULL)
public class ResponseOrder {
    private String productId;
    private Integer qty;
    private Integer unitPrice;
    private Integer totalPrice;
    private Date createdAt;

    private String orderId;
}

```

- **Service 추가**

```

public interface OrderService {
    OrderDto createOrder(OrderDto orderDetails);
    OrderDto getOrderById(String orderId);
    Iterable<OrderEntity> getOrdersByUserId(String userId);
}

```

```

@Service
public class OrderServiceImpl implements OrderService {
    OrderRepository orderRepository;

    @Autowired
    public OrderServiceImpl(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
    }

    @Override
    public OrderDto createOrder(OrderDto orderDto) {
        orderDto.setOrderId(UUID.randomUUID().toString());
        orderDto.setTotalPrice(orderDto.getQty() * orderDto.getUnitPrice());

        ModelMapper mapper = new ModelMapper();
        mapper.getConfiguration().setMatchingStrategy(MatchingStrategies.STRICT);
        OrderEntity orderEntity = mapper.map(orderDto, OrderEntity.class);

        orderRepository.save(orderEntity);

        OrderDto returnValue = mapper.map(orderEntity, OrderDto.class);

        return returnValue;
    }

    @Override
    public OrderDto getOrderById(String orderId) {
        OrderEntity orderEntity = orderRepository.findById(orderId);
        OrderDto orderDto = new ModelMapper().map(orderEntity, OrderDto.class);

        return orderDto;
    }

    @Override
    public Iterable<OrderEntity> getOrdersByUserId(String userId) {
        return orderRepository.findById(userId);
    }
}

```

- Controller 추가

```

@RestController
@RequestMapping("/order-service")
@Slf4j
public class OrderController {
    Environment env;
    OrderService orderService;

    @Autowired
    public OrderController(Environment env, OrderService orderService) {
        this.env = env;
        this.orderService = orderService;
    }
}

```



```

@GetMapping("/health-check")
public String status() {
    return String.format("It's Working in Order Service on LOCAL PORT %s (SERVER PORT %s)",
        env.getProperty("local.server.port"),
        env.getProperty("server.port"));
}

@PostMapping("/{userId}/orders")
public ResponseEntity<ResponseOrder> createOrder(@PathVariable("userId") String userId,
    @RequestBody RequestOrder orderDetails) {
    log.info("Before add orders data");
    ModelMapper mapper = new ModelMapper();
    mapper.getConfiguration().setMatchingStrategy(MatchingStrategies.STRICT);

    OrderDto orderDto = mapper.map(orderDetails, OrderDto.class);
    orderDto.setUserId(userId);
    /* jpa */
    OrderDto createdOrder = orderService.createOrder(orderDto);
    ResponseOrder responseOrder = mapper.map(createdOrder, ResponseOrder.class);

    log.info("After added orders data");
    return ResponseEntity.status(HttpStatus.CREATED).body(responseOrder);
}

@GetMapping("/{userId}/orders")
public ResponseEntity<List<ResponseOrder>> getOrder(@PathVariable("userId") String userId) throws
Exception {
    log.info("Before retrieve orders data");
    Iterable<OrderEntity> orderList = orderService.getOrdersByUserId(userId);

    List<ResponseOrder> result = new ArrayList<>();
    orderList.forEach(v → {
        result.add(new ModelMapper().map(v, ResponseOrder.class));
    });

    log.info("Add retrieved orders data");

    return ResponseEntity.status(HttpStatus.OK).body(result);
}
}

```

Orders Microservice - 기능 구현 (2)

- API Gateway Service - Order 라우트 추가

```

spring:
  application:
    name: apigateway-service
  cloud:
    gateway:
      server:
        webflux:
          default-filters:
            - name: GlobalFilter
              args:
                baseMessage: Spring Cloud Gateway WebFlux Global Filter
                preLogger: true
                postLogger: true
          routes:
            - id: user-service
              uri: lb://USER-SERVICE
              predicates:
                - Path=/user-service/**
            - id: catalog-service
              uri: lb://CATALOG-SERVICE
              predicates:
                - Path=/catalog-service/**
            - id: order-service
              uri: lb://ORDER-SERVICE
              predicates:
                - Path=/order-service/**
            - id: first-service
              uri: lb://MY-FIRST-SERVICE
              predicates:
                - Path=/first-service/**
              filters:
                - AddRequestHeader=f-request, 1st-request-header-by-yaml
                - AddResponseHeader=first-response, 1st-response-header-from-yaml
                - CustomFilter
            - id: second-service
              uri: lb://MY-SECOND-SERVICE
              predicates:
                - Path=/second-service/**
              filters:
                - AddRequestHeader=s-request, 2nd-request-header-by-yaml
                - AddResponseHeader=s-response, 2nd-response-header-from-yaml

```

서버 실행 및 API 테스트

실행 순서 : Eureka Server → API Gateway Service → User Service, Catalog Service, Order Service

- 사용자 추가

POST localhost:8000/user-service/users Send

Params Authorization Headers (10) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```

1 {
2   "email": "kg5988@naver.com",
3   "name": "KIM",
4   "pwd": "12345678"
5 }

```

Body Cookies Headers (9) Test Results 201 Created 1.10 s 387 B Save Response

JSON Preview Visualize

```

1 {
2   "email": "kg5988@naver.com",
3   "name": "KIM",
4   "userId": "4dd3a195-550f-4a0a-bcf9-654b1abf1d13"
5 }

```

- 사용자 조회

GET localhost:8000/user-service/users/4dd3a195-550f-4a0a-bcf9-654b1abf1d13 Send

Params Authorization **Headers (10)** Body Scripts Settings Cookies

Headers 9 hidden

	Key	Value	Description		Bulk Edit	Presets
☒	Authorization	Bearer eyJhbGciOiJIUzI1NiJ9.eyJlbWFpbCI6InV...				
	Key	Value	Description			

Body Cookies Headers (9) Test Results 200 OK 308 ms 394 B Save Response

JSON Preview Visualize

```

1 {
2   "email": "kg5988@naver.com",
3   "name": "KIM",
4   "userId": "4dd3a195-550f-4a0a-bcf9-654b1abf1d13",
5   "orders": []
6 }

```

- 상품 조회

GET http://localhost:8000/catalog-service/catalogs Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

	Key	Value	Description	⋮ Bulk Edit
	Key	Value	Description	

Body Cookies Headers (3) Test Results 200 OK 261 ms 491 B Save Response

{ } JSON Preview Visualize

```

1  [
2    {
3      "productId": "CATALOG-001",
4      "productName": "Berlin",
5      "unitPrice": 1500,
6      "stock": 100,
7      "createdAt": "2025-10-15T16:01:38.385+00:00"
8    },
9    {
10     "productId": "CATALOG-002",
11     "productName": "Tokyo",
12     "unitPrice": 1000,
13     "stock": 110,
14     "createdAt": "2025-10-15T16:01:38.386+00:00"
15   },
16   {
17     "productId": "CATALOG-003",
18     "productName": "Stockholm",
19     "unitPrice": 2000,
20     "stock": 120,
21     "createdAt": "2025-10-15T16:01:38.387+00:00"
22   }
23 ]

```

- 상품 주문

POST http://localhost:8000/order-service/4dd3a195-550f-4a0a-bcf9-654b1abf1d13/orders Send

Params Authorization Headers (8) Body Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON Schema Beautify

```

1  {
2    "productId": "CATALOG_0002",
3    "qty": "5",
4    "unitPrice": 1000
5  }

```

Body Cookies Headers (3) Test Results 201 Created 854 ms 241 B Save Response

{ } JSON Preview Visualize

```

1  {
2    "productId": "CATALOG_0002",
3    "qty": 5,
4    "unitPrice": 1000,
5    "totalPrice": 5000,
6    "orderId": "9d493e66-5bba-4d6c-a90d-1d8728efedc0"
7  }

```

- 사용자 별 주문 조회

GET

http://localhost:8000/order-service/4dd3a195-550f-4a0a-bcf9-654b1abf1d13/orders

Send

ParamsAuthorizationHeaders (8)Body •ScriptsSettingsCookies

Query Params

	Key	Value	Description	⋮ Bulk Edit
	Key	Value	Description	

BodyCookiesHeaders (3)Test Results🕒200 OK • 164 ms • 282 B🌐📄 Save Response⋮

{ } JSON ▾▶ Preview🖼 Visualize ▾

1[
2 {
3 "productId": "CATALOG_0002",
4 "qty": 5,
5 "unitPrice": 1000,
6 "totalPrice": 5000,
7 "createdAt": "2025-10-15T16:13:45.026+00:00",
8 "orderId": "9d493e66-5bba-4d6c-a90d-1d8728efedc0"
9 }
10]